

Concept 11: Cron Jobs & Task Scheduling

Why This Matters

Your boss mentioned two ways to run the reminder batch: an in-app infinite loop or a cron job. These are two fundamentally different approaches to **scheduling recurring tasks**, and understanding both (plus when to use each) is essential for any backend developer.

What Is a Cron Job?

A **cron job** is a task scheduled by the operating system (Linux/Unix) to run automatically at specified intervals. The word "cron" comes from "chronos" (Greek for time).

It's an **external scheduler** — it lives outside your application. The OS wakes up, runs your script, and the script exits. Your application doesn't need to be running for the cron job to work (though it does need the database to be accessible).

Cron Syntax



The diagram illustrates the Cron syntax fields with brackets and labels:

- minute (0-59)
- hour (0-23)
- day of month (1-31)
- month (1-12)
- day of week (0-7, 0 and 7 = Sunday)

The example cron job is: `* * * * * command_to_run`

Common Examples

bash

```
# Every 5 minutes
*/5 * * * * python /app/scripts/check_reminders.py

# Every hour at minute 0
0 * * * * python /app/scripts/check_reminders.py

# Every day at 8:00 AM
```

```
0 8 * * * python /app/scripts/daily_report.py

# Every Monday at 9:00 AM
0 9 * * 1 python /app/scripts/weekly_digest.py

# Every first day of the month at midnight
0 0 1 * * python /app/scripts/monthly_archive.py
```

Setting Up a Cron Job

bash

```
# Edit the cron table
crontab -e

# Add a line (runs check_reminders every 5 minutes)
*/5 * * * * cd /app && python scripts/check_reminders.py >> /var/log/reminders.log 2>&1

# View current cron jobs
crontab -l
```

The `>> /var/log/reminders.log 2>&1` part redirects both output and errors to a log file — essential for debugging since cron runs silently.

In-App Scheduling (Infinite Loop with Sleep)

Instead of relying on the OS, the scheduling lives **inside your application**:

python

```
import time
from datetime import datetime

def check_reminders():
    """Find and process all due reminders"""
    due_reminders = db.query(Reminder).filter(
        Reminder.trigger_date <= datetime.now(),
        Reminder.status == 'pending'
    ).all()

    for reminder in due_reminders:
        send_notification(reminder)
        reminder.status = 'sent'
    db.commit()

# The infinite loop
while True:
    try:
        check_reminders()
    except Exception as e:
```

```
log.error(f"Batch error: {e}")
time.sleep(300) # Sleep 5 minutes
```

This runs as a **thread or background process** within your app. The app itself is the scheduler.

More Sophisticated: APScheduler

Python has libraries that provide in-app scheduling without manual loops:

python

```
from apscheduler.schedulers.background import BackgroundScheduler
```

```
scheduler = BackgroundScheduler()
scheduler.add_job(check_reminders, 'interval', minutes=5)
scheduler.start()
```

Comparison

Criteria	Cron Job (External)	In-App Loop	In-App Scheduler (APScheduler)
Independence	Runs even if app is down	Dies with the app	Dies with the app
Setup complexity	Needs OS-level access	Just code	Library dependency
Monitoring	OS-level logs, separate monitoring	Application logs	Application logs
Deployment	Must configure cron on server	Deploys with app	Deploys with app
Granularity	Minimum 1 minute	Any interval	Any interval
Error handling	Script must handle its own errors	Integrated with app error handling	Built-in retry/error support
Multiple instances	Risk of duplicate runs if multiple servers	Risk of duplicate runs if multiple instances	Same risk
Best for	Production systems, DevOps maturity	Quick prototypes, demos	Production apps without cron access

Which to Use for Your PFE

For the demo/defense: In-app scheduling (APScheduler or simple loop) is simpler. It deploys with your app, needs no server configuration, and you can show it working in real-time.

For a production argument in your report: Mention cron jobs as the production-grade approach, explaining that separating the scheduler from the app provides better resilience and monitoring.

The honest answer for your defense: "We used in-app scheduling for the prototype to simplify deployment. In production, this would be migrated to a cron job or a dedicated task queue (like Celery) for resilience and independent scaling."

Key Vocabulary

Term	Definition
Cron	A Unix/Linux system daemon that runs scheduled tasks at specified intervals
Crontab	The configuration file where cron jobs are defined (cron table)
Daemon	A background process that runs continuously on a system
Background thread	A thread in your application that runs concurrently with the main process
APScheduler	A Python library for in-process task scheduling
Celery	A distributed task queue for Python, used for both scheduling and async task processing
Idempotent batch	A batch process that produces the same result whether run once or multiple times